Research Problem: 单卡 MoE 推理的存储需求

- MoE 每次仅激活少数专家，但不同输入可能激活不同专家。单个推理任务可用的显存和 CPU 内存有限，无法容纳全部专家；当所选专家位于 HDD 时，必须读取并传输权重后才能计算。

单卡仍需支持模型中的全部专家

本次路由	E1	E2	E3	E4	E5	E6	E7	**E8**

蓝色 E8 为本次激活的专家；其存储位置决定读取开销。

GPU 显存	E1	E2		直接计算
CPU 内存	E3	E4	E5	传输至显存
HDD	E6	E7	**E8**	从 HDD 读取并传输

E8：从 HDD 读取 → 经 CPU 内存传输 → 在 GPU 上计算

应用场景与研究任务

共享 GPU 服务器
任务仅分配一张 GPU 和部分内存

离线生成
问答、代码生成允许更长响应时间

单卡工作站
权重规模超过任务可用内存

MECC：MoE 专家缓存编译器
MoE Expert Cache Compiler
初始放置 • 联合缓存 • 动态调整

研究目标：在单卡上运行完整 MoE 模型，降低专家权重读取造成的延迟。

Related Work：现有方法与 MECC 的对比

- 现有方法支持跨显存、CPU 内存和 HDD 读取专家，并缓存常用专家。在共享服务器中，还需要根据任务的内存预算确定初始放置、提前加载常用专家，并适应任务变化。

方法	显存/内存/HDD 三级存储	完整专家计算	按内存预算安排专家	提前加载常用专家	预测 HDD 读取需求	任务变化时更新缓存
SwapMoE[1]	✗	✗	✗	✓	✗	✓
Fiddler[2]	✗	✓	✗	✗	✗	✗
MoE-Infinity[3]	✗	✓	✗	✗	✗	✓
MoE-Lightning[4]	✗	✓	✗	✗	✗	✗
SSD-LLaMA[5]	✓	✓	✗	✗	✗	✓
MECC	✓	✓	✓	✓	✓	✓

表内主要比较单卡卸载方法；MoE-Lightning 也测试多卡。8 卡 FlashMoE 是分布式算子参照，不是同机单卡基线。

Our Approach: MECC 的三项设计

- 部署前统计专家的使用频率，据此确定显存和 CPU 内存中的初始专家。推理过程中根据实际路由更新缓存。下图依次展示三项设计如何减少 HDD 读取。

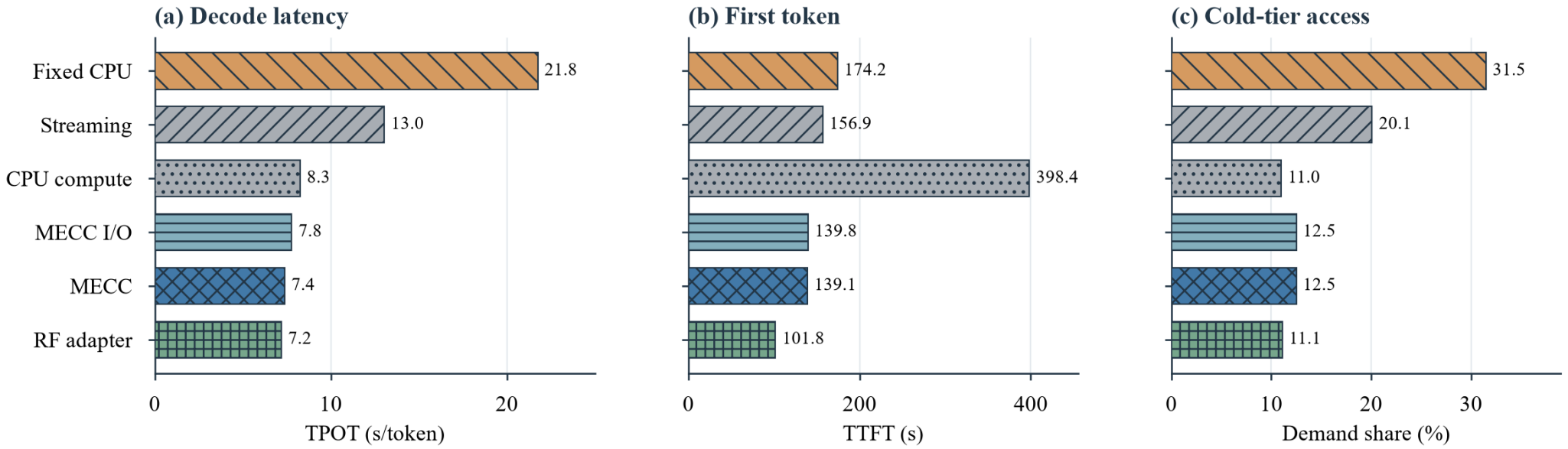


深蓝表示当前高频访问或被选中的专家；其他方块表示专家所在的存储层级。

HDD 保存完整专家权重；显存与 CPU 内存缓存当前更可能访问的专家。

Motivation：冷存储访问进入解码路径

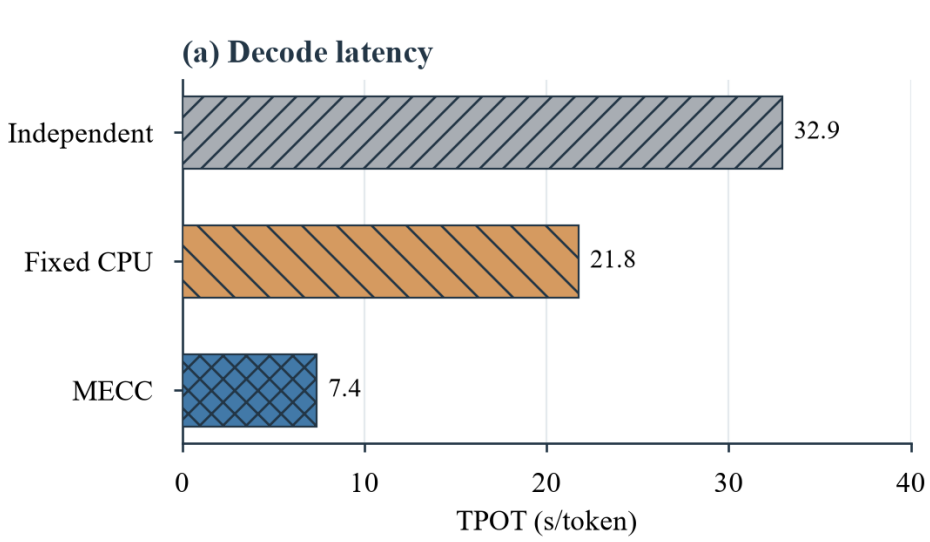
- 单卡运行时，所选专家若不在显存或 CPU 内存，就要从冷存储读取。
  相同模型与输入下，不同缓存策略的访问比例和每 token 延迟同时变化。

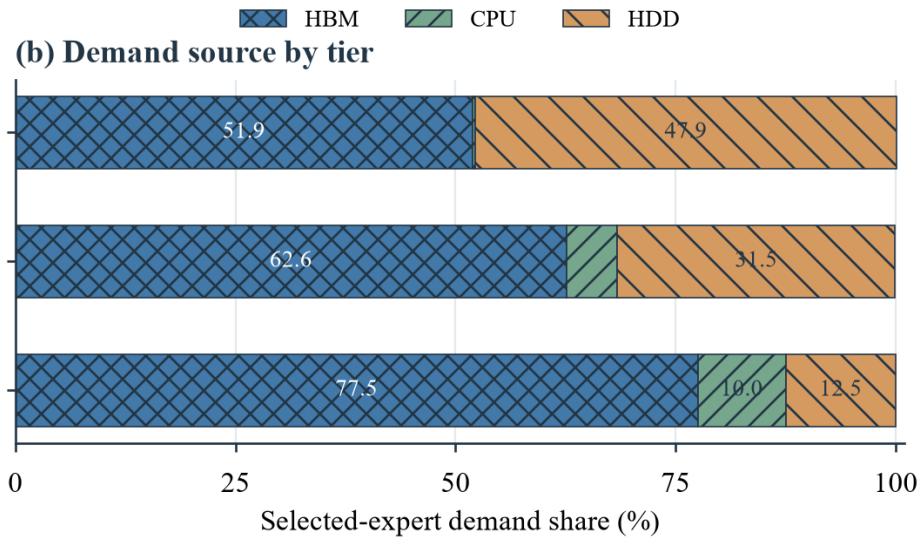


主对比：六种本地策略均使用一张 A100；右图为全阶段需求比例。

Insight：联合更新两级缓存

- 访问 CPU 内存中的专家后，MECC 将其提升至显存，并把显存中被替换的专家保留在 CPU 内存。同配置实验比较独立缓存、固定 CPU 缓存和联合更新。

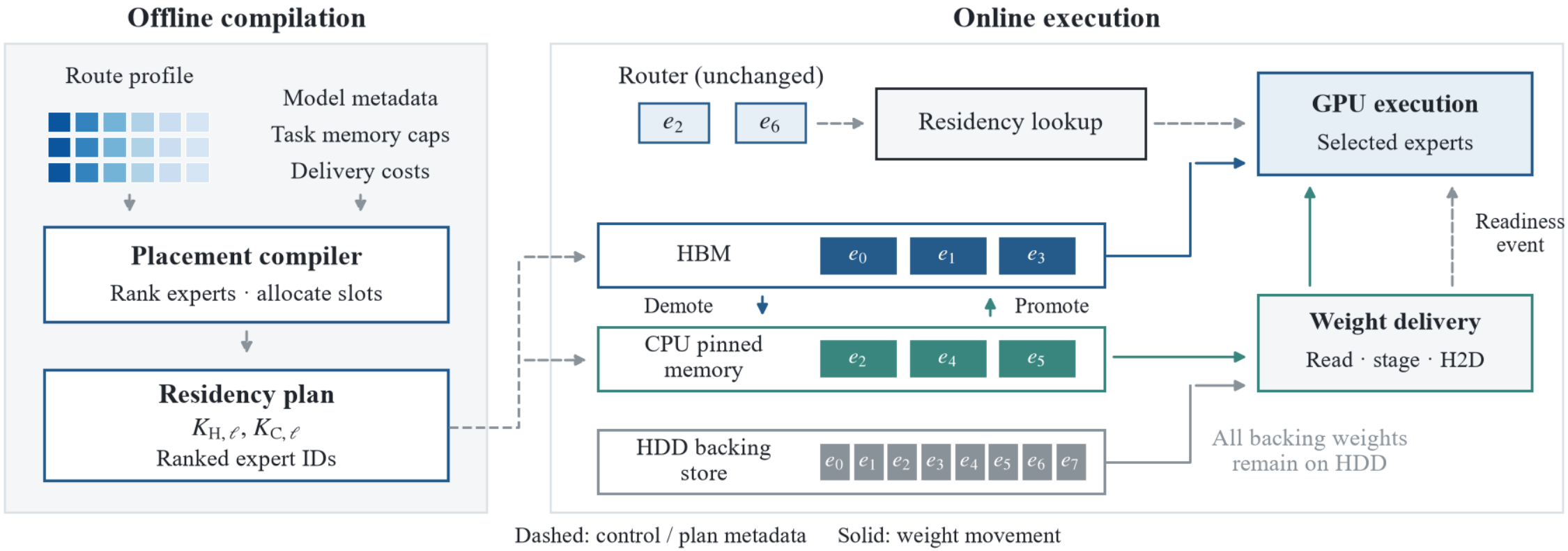




联合更新的 TPOT 为 7.38 秒/token；独立缓存为 32.94 秒/token。

System Overview: MECC 系统总览

- MECC 在部署前确定专家的初始位置，在推理时根据实际路由访问并更新缓存。HDD 保存完整的专家权重；显存和 CPU 内存只缓存部分专家。所选专家的权重到达显存后执行计算。



部署前确定初始位置；推理时根据路由访问和更新缓存。

Evaluation Setup：单卡真实推理

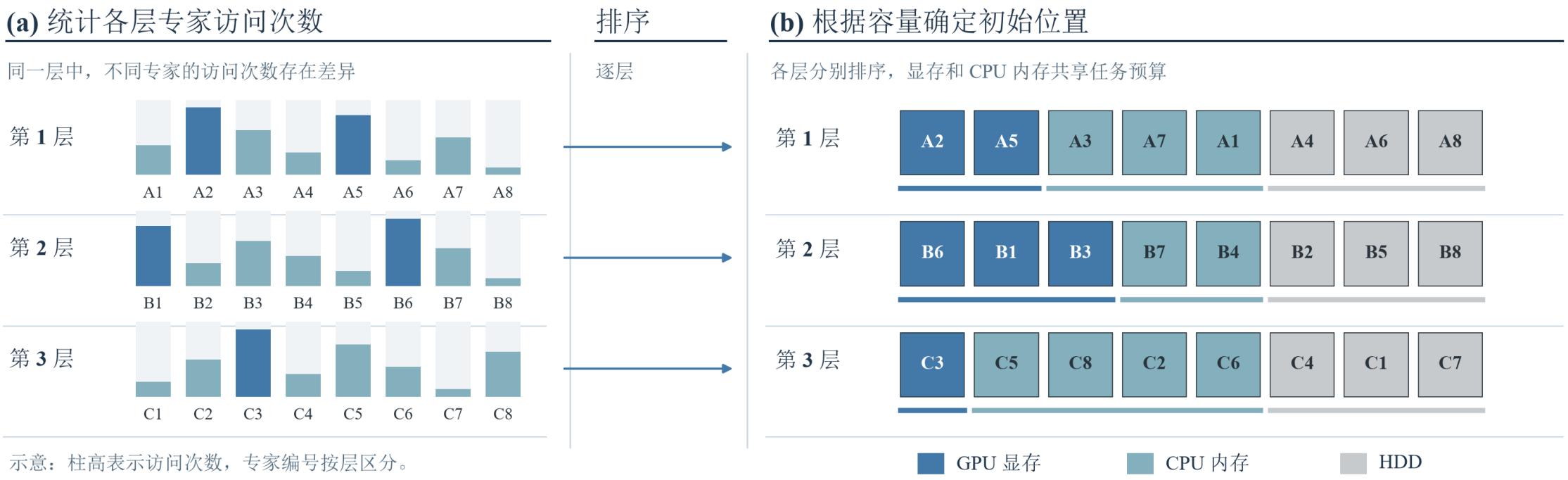
- 主对比使用 Mixtral-8×7B、A100 80 GB 和系统报告为旋转介质的 RAID 逻辑盘。六种策略在相同模型、相同名义缓存槽位和相同输入规模下各运行一次。

模型与设备	Mixtral-8×7B • 单张 A100 • FP16 权重
输入与输出	WikiText-2 测试集 • 8 条输入 • 每条生成 64 token
缓存配置	每层显存 4 个槽位 • CPU 内存 3 个槽位 • 其余冷存储
同机单卡对照	流式读取 • 固定 CPU 缓存 • 独立缓存 • MECC • RF 本地适配版

下列图只使用已提交的 A100 result.json；每个柱子对应一次完整运行。

Method I：确定专家的初始存储位置

- 部署前使用代表性输入统计每层专家的访问频率，并记录权重大小及任务的内存预算。
  编译器按层排序，在显存和 CPU 内存预算内确定初始位置；HDD 保存完整权重。



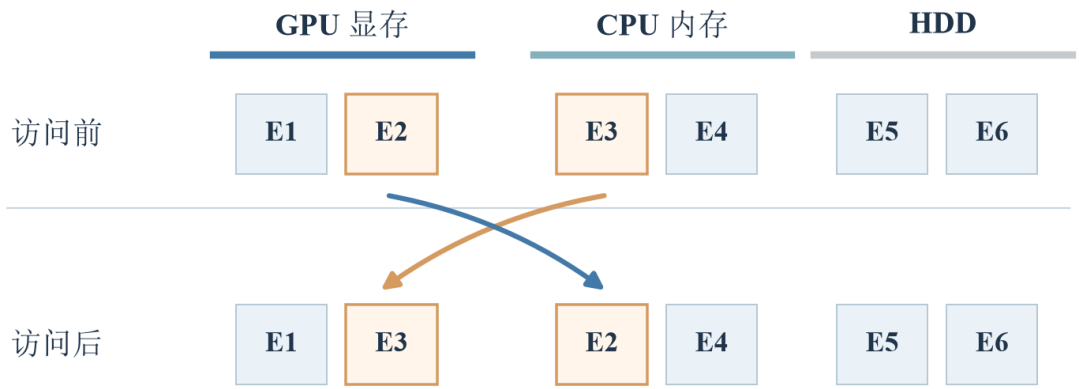
编译结果记录各层的缓存容量和专家编号；推理时继续根据实际请求更新缓存。

Method II：更新缓存并执行专家计算

- 推理时先根据路由查找所选专家。CPU 内存命中时将专家传输到显存，并把被替换的专家保留在 CPU 内存；访问 HDD 时，等待权重到达显存后再执行计算。

(a) 一次 CPU 内存访问如何更新缓存

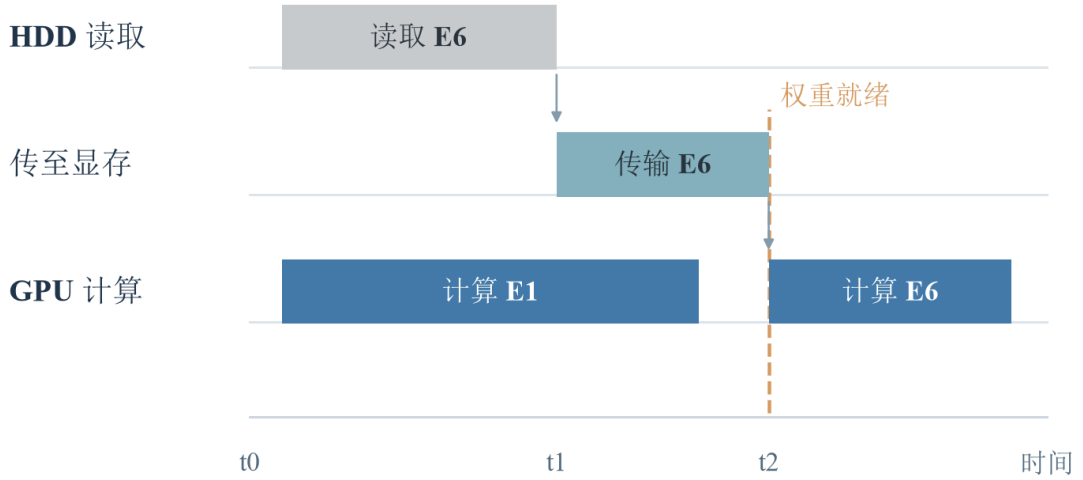
访问 E3 后，E3 进入显存；被替换的 E2 保留在 CPU 内存



E2 保留在 CPU 内存，后续访问无需再次读取 HDD。

(b) 所选专家何时可以计算

已在显存的 E1 先计算；E6 的权重到达后再执行

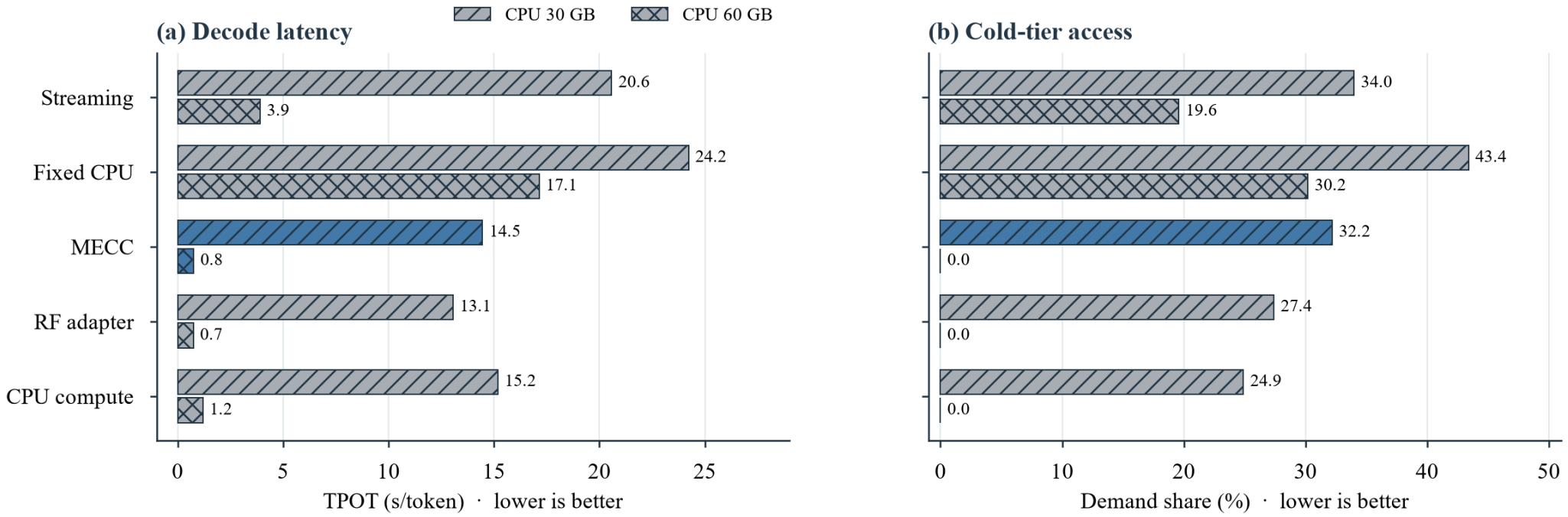


同一请求中，权重读取与已有专家的计算可以重叠。

两级缓存更新减少重复读取；已在显存的专家可先计算，其他专家在权重到达后计算。

Results: CPU 内存槽位的影响

- 同一组 WikiText-2 输入下，将名义 CPU 预算从 30 GB 增至 60 GB，
  每层可缓存的专家从 2 个增至 4 个；五种策略的解码延迟与冷存储访问随之变化。



30/60 GB 为同输入规模；20 GB 的输入来源不同，未并入这张图。

Conclusion：单卡缓存更新的实测收益

- 在本次单卡 Mixtral-8×7B 测试中，MECC 的联合缓存更新减少了冷存储访问，解码延迟低于流式读取和固定 CPU 缓存；RF 适配版在同一主测试中略快。

2.95×

相对固定 CPU 缓存的 TPOT 比

1.77×

相对未优化流式读取的 TPOT 比

12.5%

MECC 的全阶段冷存储需求比例

结果基于统一初始放置；逐层画像和跨任务切换仍需独立实验。